

Fatigue Routine v7_2

Repository Overview and Help

2026-02-25

1 What this repository does (high-level)

This repository evaluates fatigue assessment methods on experimental datasets stored in Excel files (one workbook per material). The workflow is:

1. **Run one method on one material** with `1_run_material.py`:

- import the Excel dataset;
- build a *design subset* (uniaxial $R \approx -1$ points) and fit a power law (Basquin-like) correlation between the method “damage parameter” DP and cycles to failure N_f ;
- evaluate the selected method on the remaining cases;
- export a per-method CSV into `Results/`.

2. **Compare methods and generate publication-ready plots** with `2_compare_results.py`:

- reads all CSVs under `Results/`;
- generates PNG and TikZ/PGFPlots figures under `Results/Comparison/`;
- writes a single global legend/statistics table (LaTeX + CSV) and a single style file `Results/Comparison/faber_plot_styles.tex`.

Important v7_2 feature: automatic metric switch for “flat” calibration curves. If the fitted calibration curve is almost horizontal (slope parameter $k > 15$), predicting $N_{f,e}$ by inverting the fit becomes meaningless. In that case, the code automatically switches to a DP-based comparison:

- Standard mode: $Error_{\ln} = \ln \left(\frac{N_f}{N_{f,e}} \right)$.
- Flat-slope mode: $Error_{\ln,dp} = \ln \left(\frac{DP}{DP_{fit}(N_f)} \right)$.

The chosen mode is written in the CSV column `Metric_mode`, so `2_compare_results.py` automatically generates the correct plots.

2 Quick start (from zero)

2.1 Install dependencies

Required Python packages (tested with Python 3.10+):

- `numpy`
- `pandas`

- `openpyxl` (Excel import)
- `matplotlib` (plots)

Install with:

```
pip install numpy pandas openpyxl matplotlib
```

2.2 Run one method on one material

Edit the configuration block at the top of `1_run_material.py`:

- `MATERIAL_NAME`: must match the Excel file naming convention used by the importer.
- `MODEL_NAME`: one of the names registered in `fatigue/models/registry.py`. Use suffix `_ext` for CP “extreme” variant.

Then run:

```
python 1_run_material.py
```

Output: a CSV under `Results/` named `<METHOD>_<MATERIAL>.csv`.

2.3 Compare all methods

Once you have multiple CSVs in `Results/`, run:

```
python 2_compare_results.py
```

Output folder: `Results/Comparison/`.

3 Folder structure

Simplified repository tree (Python sources only; `__pycache__` omitted):

```
|-- 0_calibrate_material_params.py
|-- 1_run_material.py
|-- 2_compare_results.py
|-- Experimental_Data/
|-- Results/
'-- fatigue/
    |-- eval/
    |-- imp/
    |-- models/
    |   |-- cp_methods/
    |   |-- energy_based_methods/
    |   '-- invariant_methods/
    |-- planes/
    '-- report/
```

Notes.

- `Experimental_Data/` contains one Excel file per material.
- `Results/` is where `1_run_material.py` writes per-method CSV outputs. The comparison script writes under `Results/Comparison/`.
- The main implementation lives in `fatigue/` (import, plane sampling, model registry, evaluator, export).

4 What each file is for

Top-level scripts

- `1_run_material.py`: Main entry point for fatigue evaluation on one selected material and one selected model. Orchestrates: plane sampling → Excel import → design-subset selection → calibration (Basquin fit) → evaluation → plots → CSV export into `Results/`.
- `2_compare_results.py`: Post-processing entry point. Reads CSV files from `Results/`, detects the error metric from `Metric_mode`, compares methods per material, produces parity + distribution plots, and exports both PNG figures and TikZ/PGFPlots (`.tex` + `.dat`) counterparts.
- `0_calibrate_material_params.py`: optional utility to calibrate selected *material parameters* (e.g. k_{FS} , k_{FI} , etc.) by minimizing fatigue scatter on selected subsets. It does not modify Excel files; it prints suggested values.

fatigue/ package

This package implements a small plugin architecture that unifies Critical Plane (CP) and non-CP (direct) models.

Import and dataset utilities (fatigue/imp/).

- `fatigue/imp/experimental_data.py`: Excel import and dataset utilities. Reads material properties from the `Summary` sheet (row specified by the named cell `Average`), reads experimental points from the `Dataset` sheet, filters invalid cases (Notched/Run-out), builds tensors at max/min load, and (when phase columns exist) builds harmonic coefficients (mean/sin/cos) to support out-of-phase loading.

Plane sampling (fatigue/planes/).

- `fatigue/planes/sampling.py`: Generates the list of candidate plane orientations (rotation matrices R_i). Convention: the 3rd column of R_i is the plane normal (local z' axis).

Models and registry (fatigue/models/).

- `fatigue/models/base.py`: Shared dataclasses and protocols: `FatigueModel`, `CriticalPlaneMethod`, `CaseResult`, `CPPlaneResult`, etc.
- `fatigue/models/registry.py`: Central registry of all case-level models exposed to the user. Wires CP methods (wrapped through `CPModelAdapter`) and direct models into `get_models(names)`.
- `fatigue/models/cp_adapter.py`: Wraps any plane-level CP method into a case-level `FatigueModel`. Provides standard (`"`) and optional extreme (`"_ext"`) variants.

Plane-level CP method implementations live in `fatigue/models/cp_methods/`. The registry currently includes legacy CP methods (FS, FIN, SWT, DangVan, CarSpa, Mataka, McDiarmid) and additional CP methods added during the project.

Direct (non-CP) models live in `fatigue/models/invariant_methods/` and `fatigue/models/energy_based_methods/`. Examples: Von Mises equivalent stress (VM), Ottosen reduced equivalent stress (OTT), SWTD, ZhuEDP.

Evaluation and fitting (fatigue/eval/).

- `fatigue/eval/evaluator.py`: Critical-plane evaluator: rotates tensors to each candidate plane, optionally applies harmonic (phase-shift) corrections (ellipse-aware shear range + harmonic extrema for normal components), then evaluates the CP method and returns best-by-metric and extreme values.

- `fatigue/eval/calibration.py`: Builds the design-series (uniaxial $R \approx -1$) for each model, fits Basquin-like parameters ($CP = AN^b$) and prepares prediction mappings.
- `fatigue/eval/fitting.py`: Numerical fitting utilities (log-log power-law fit, survival shift, etc.).
- `fatigue/eval/fat_cycles.py`: Basquin inversion $N(CP)$ and plotting utilities.

Reporting (`fatigue/report/`).

- `fatigue/report/export_csv.py`: CSV export utility. Writes per-method per-material CSV files with:
 - `Method_family` (inferred from module path / registry),
 - `Loading_type` (Tensile / Torsion / Multiaxial proportional / Multiaxial non-proportional),
 - `Metric_mode` (Nf or DP_DIFF),
 - both error columns `Error_ln` and `Error_ln_dp`.

5 Results and CSV schema

Each run of `1_run_material.py` writes one CSV file:
`Results/<METHOD>_<MATERIAL>.csv`.

5.1 Meaning of the key columns

- `Material`, `Method`: identifiers.
- `Method_family`: used by `2_compare_results.py` to build per-family color palettes.
- `Loading_type`: classified per datapoint using stress components and phase columns.
- `Flag`: 0 = design point (used for calibration); 1 = other point.
- `Metric_mode`:
 - `Nf`: standard mode (predict $N_{f,e}$).
 - `DP_DIFF`: flat-slope mode (compare DP against the fitted curve at experimental N_f).
- `Nf_exp`: experimental cycles to failure N_f .
- `Nf_expected`: expected cycles to failure $N_{f,e}$ (only meaningful in `Nf` mode).
- `CP_value`: damage parameter computed by the method (historical name: `CP`).
- `DP_fit`: calibration curve value $DP_{fit}(N_f)$.
- `Error_ln`: $\ln\left(\frac{N_f}{N_{f,e}}\right)$ (only meaningful in `Nf` mode).
- `Error_ln_dp`: $\ln\left(\frac{DP}{DP_{fit}(N_f)}\right)$ (only meaningful in `DP_DIFF` mode).

How `Loading_type` is inferred (rule of thumb). The importer stores stress harmonics per case (mean/sin/cos) and optional phase shifts (`Phase radial load` and `Phase torsional load` columns in the Excel `Dataset` sheet). The runner classifies each datapoint as:

- **Tensile**: only normal stress components are present (axial and/or radial), torsion is absent.
- **Torsion**: only shear component is present.

- **Multiaxial proportional:** at least two components are present and all relevant phase shifts are 0° .
- **Multiaxial non-proportional:** at least two components are present and at least one relevant phase shift is non-zero.

6 Comparison outputs (plots + TikZ)

2.compare_results.py writes under Results/Comparison/:

- one folder per material: Results/Comparison/<MATERIAL>/
- publication-ready TikZ files (plus .dat numeric tables) in tikz_data/
- a single global style file: Results/Comparison/faber_plot_styles.tex
- a single global legend/statistics table: Results/Comparison/summary_error_stats_table.tex

Including figures in a LaTeX article. Copy the whole Results/ folder into your article project and include the TikZ file, for example:

```
\input{Results/Comparison/AISI316L/AISI316L_expected_vs_experimental.tex}
```

The TikZ files already start as:

```
\begin{tikzpicture}
\input{Results/Comparison/faber_plot_styles.tex}
\begin{axis}[
```

so you do not need to input the style file manually.

Underscores in paths. Data file paths inside TikZ use `_` (not `\_`) so they work as filesystem paths. Method names in legend entries are LaTeX-escaped (e.g. `FIN_ext`).

7 Key data structures

7.1 Experimental data dictionary

fatigue/imp/experimental_data.py returns a nested structure:

```
data[material_name] = {
    "material": {...},
    "cases": {
        case_id: {
            "meta": {"Nf_cycles": ..., "phase_rad_deg": ..., "phase_tor_deg": ...},
            1: { 1: {"stress": Smax, "strain": Emax},
                2: {"stress": Smin, "strain": Emin} },
            "harmonics": {"S_mean":..., "S_sin":..., "S_cos":...,
                          "E_mean":..., "E_sin":..., "E_cos":...}
        },
        ...
    }
}
```

Legacy convention (kept). Loadstep 1 = “max” and loadstep 2 = “min”. This interface is preserved for all existing CP methods.

7.2 Model outputs and variants

Each case-level model returns `CaseResult(values={...})`. Variants are:

- `""`: standard value (used for calibration and default evaluation)
- `"_ext"`: optional extreme value (for CP methods that expose an “extreme” variant through the CP adapter)

7.3 Plane sampling

Plane orientations are passed as `R_list`, a list/array of rotation matrices R_i . Convention: the 3rd column of R_i is the candidate plane normal (local z' axis).

8 In-phase and out-of-phase loading (phase shift handling)

8.1 Phase input columns

Out-of-phase loading is enabled through two optional columns in the Excel `Dataset` sheet:

- `Phase radial load (deg)`: phase of radial loading with respect to axial
- `Phase torsional load (deg)`: phase of torsional loading with respect to axial

If these columns are missing, phases default to 0 (in-phase).

8.2 Single-harmonic representation stored per case

All load inputs are assumed sinusoidal with a single frequency ω :

$$q(t) = q_{\text{mean}} + q_a \sin(\omega t + \phi)$$

To support phase shifts while keeping the legacy 2-loadstep interface, each case stores a compact harmonic decomposition:

$$\sigma(t) = \sigma_{\text{mean}} + \sigma_{\text{sin}} \sin(\omega t) + \sigma_{\text{cos}} \cos(\omega t),$$

(and similarly for $\varepsilon(t)$). This is built via:

$$q_a \sin(\omega t + \phi) = (q_a \cos \phi) \sin(\omega t) + (q_a \sin \phi) \cos(\omega t).$$

Axial loading is the reference ($\phi = 0$), therefore its amplitude contributes only to the `sin` term.

8.3 Ellipse-aware shear range and 2-loadstep preservation

Critical-plane methods in this repository take two rotated loadsteps ($S0_r, E0_r$) and ($S1_r, E1_r$). For out-of-phase loading, the in-plane shear traction vector on a candidate plane is:

$$\tau(t) = v_{\text{sin}} \sin(\omega t) + v_{\text{cos}} \cos(\omega t), \quad v_{\text{sin}} = \begin{bmatrix} \tau_{xz, \text{sin}} \\ \tau_{yz, \text{sin}} \end{bmatrix}, \quad v_{\text{cos}} = \begin{bmatrix} \tau_{xz, \text{cos}} \\ \tau_{yz, \text{cos}} \end{bmatrix}.$$

For single-harmonic loading, $\tau(t)$ traces an ellipse. The code defines the equivalent shear amplitude as the vector sum of the major and minor ellipse semi-axes, which reduces to:

$$\tau_{a, \text{eq}} = \sqrt{\|v_{\text{sin}}\|^2 + \|v_{\text{cos}}\|^2}, \quad \Delta\tau_{\text{eq}} = 2\tau_{a, \text{eq}}.$$

In `fatigue/eval/evaluator.py`, the rotated max/min tensors are overridden only in the plane shear components (xz, yz) so that the two-step shear range computed from ($S0_r, S1_r$) equals $\Delta\tau_{\text{eq}}$. Mean shear is preserved; the shear direction is chosen robustly from v_{sin} (fallback to v_{cos}). The same procedure is applied to shear strain.

8.4 Normal scalar extrema under phase shift

For any scalar component:

$$q(t) = q_{\text{mean}} + q_{\text{sin}} \sin(\omega t) + q_{\text{cos}} \cos(\omega t),$$

the amplitude is $q_a = \sqrt{q_{\text{sin}}^2 + q_{\text{cos}}^2}$ and extrema are $q_{\text{mean}} \pm q_a$. The evaluator updates the plane-normal scalar components (default zz in the plane frame) accordingly for both stress and strain.

Important limitation. Direct (rotation-invariant) models currently operate on the legacy two-loadstep tensors and do not use the harmonic coefficients.

9 Running the code

9.1 1_run_material.py

Edit the configuration block:

- `MATERIAL_NAME` selects the Excel file in `Experimental_Data/`
- `MODEL_NAME` selects the method/model (e.g. FS, FIN, FIN_ext, VM, SWTD)
- `MODEL_NAMES` lists which models are calibrated on the design subset

Then run:

```
python 1_run_material.py
```

9.2 2_compare_results.py

This script scans `Results/` for CSV files and writes comparison outputs in `Results/Comparison/`. Example:

```
python 2_compare_results.py --results_dir Results --out_dir Results/Comparison
```

10 Outputs

10.1 CSV format

`fatigue/report/export_csv.py` writes a lightweight CSV with columns:

Material	Method	Nf_exp	CP_value	Nf_expected	Flag	Error_ln
Flag: 0=design points, 1=other points						

where:

$$\text{Error}_{\ln} = \ln \left(\frac{N_{f,\text{exp}}}{N_{f,\text{expected}}} \right).$$

10.2 Comparison outputs

For each material, `2_compare_results.py` generates:

- boxplot of `Error_ln` across methods,
- normal PDF curves based on μ and σ of `Error_ln`,
- Expected vs Experimental scatter in log-log scale with bisector and factor lines,
- TikZ/PGFPlots `.tex` equivalents plus numeric `.dat` files under `Results/Comparison/<material>/tikz_da`